



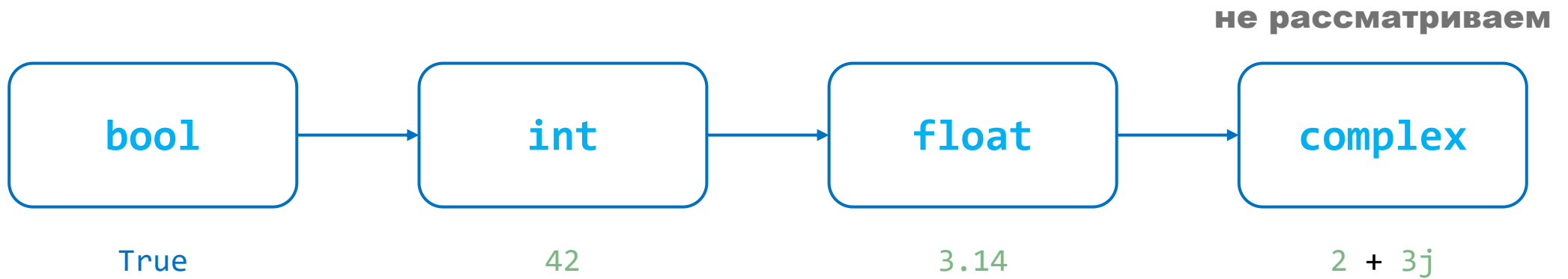
Числовые типы данных

Докладчик: Евграфов Михаил



Введение

Четыре числовых типа



**каждый следующий тип — расширение
предыдущего**

Приведение по цепочке типов

<code>int(True)</code>	<code># bool -> int</code>
<code>float(5)</code>	<code># int -> float</code>
<code>complex(5.0)</code>	<code># float -> complex</code>



Целочисленный тип

int

Десятичные литералы

```
positive_number = 42  
negative_number = -17  
million = 1_000_000
```

Системы счисления

<code>0b1010</code>	# -> 10	(двоичная)
<code>0o17</code>	# -> 15	(восьмеричная)
<code>0xFF</code>	# -> 255	(шестнадцатеричная)

<code>bin(10)</code>	# '0b1010'
<code>oct(15)</code>	# '0o17'
<code>hex(255)</code>	# '0xff'

Ограничения

```
num_bin = 0b2
```

```
# SyntaxError: invalid digit '2' in binary literal
```

```
num_oct = 0o9
```

```
# SyntaxError: invalid digit '9' in octal literal
```

```
num = 00101
```

```
# SyntaxError: leading zeros in decimal integer literals...
```

```
num_hex = 0xG
```

```
# SyntaxError: invalid hexadecimal literal
```

Замечание

```
num_bin = 0b101010
```

```
num_hex = 0x2A
```

```
print(  
    num_bin == num_hex,  
    f"bin: {num_bin}",  
    f"hex: {num_hex}",  
    sep="\n",  
)
```

```
# True
```

```
# bin: 42
```

```
# hex: 42
```

int()

<code>int(3.9)</code>	<code># 3</code>	-- отбрасывает дробь
<code>int(-3.9)</code>	<code># -3</code>	-- к нулю, не round()
<code>int(True)</code>	<code># 1</code>	
<code>int("42")</code>	<code># 42</code>	
<code>int("1010", 2)</code>	<code># 10</code>	
<code>int("ff", 16)</code>	<code># 255</code>	

int(): выбор системы счисления

Явное основание

```
int("1010", 2)    # 10
int("17", 8)       # 15
int("ff", 16)      # 255
int("z", 36)       # 35
```

base от 2 до 36

Автоопределение по префиксу

```
int("0b1010", 0)   # 10
int("0xff", 0)     # 255
```

base=0 -- по 0b / 0o / 0x

Диапазон значений

Python: int

```
huge_power = 2 ** 1000  
# никакого overflow
```

диапазон не ограничен

C / Java: int

```
int x = 2000000000;  
x = x + x;  
// overflow!
```

ограничен 32/64 битами



Тип с плавающей точкой float

Литералы float

```
positive_number = 3.14  
negative_number = -0.5
```

.5	# 0.5
5.	# 5.0
1_000.5	# 1000.5
1e3	# 1000.0
2e-3	# 0.002

Эквивалентные литералы

```
num_float1 = 42.0
```

```
num_float2 = 42.
```

```
print(  
    num_float1 == num_float2,  
    f"with part after point: {num_float1}",  
    f"without part after point: {num_float2}",  
    sep="\n",  
)  
# True  
# with part after point: 42.0  
# without part after point: 42.0
```

float()

float(5)	# 5.0
float(True)	# 1.0
float("3.14")	# 3.14
float(" 3.14 ")	# 3.14
float("abc")	# ValueError

Ограничения float

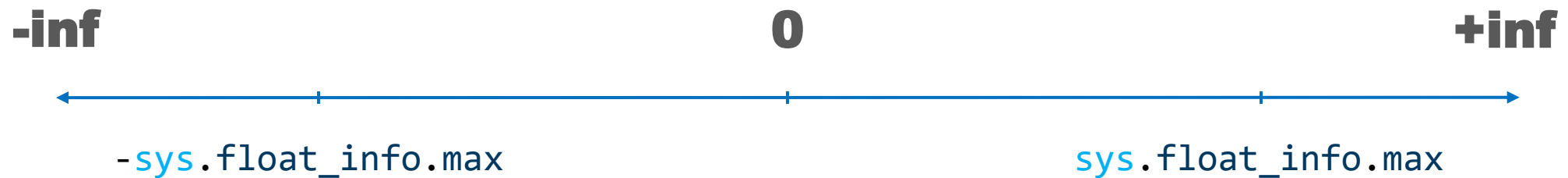
```
import sys
```

```
print(sys.float_info.max)  
# ~1.7976931348623157e+308
```

```
print(sys.float_info.min)  
# ~2.2250738585072014e-308
```

```
print(sys.float_info.epsilon)  
# ~2.22e-16
```

Числовая ось float



NaN — вне числовой оси

+inf и -inf

```
float("inf")    # inf  
float("-inf")   # -inf
```

```
import math  
math.inf        # inf  
-math.inf       # -inf
```

Бесконечность и переполнение

```
huge = sys.float_info.max
huge * 2      # inf  -- переполнение
-huge * 2     # -inf

10.0 ** 400   # OverflowError -- ** ведёт себя иначе
```

nan

```
float("nan")           # nan
math.nan               # nan

math.inf - math.inf    # nan
math.inf * 0           # nan
math.inf / math.inf    # nan

value = float("nan")
value == value          # False
math.isnan(value)       # True
```



Логический тип `bool`

True и False

```
var1 = True
var2 = True
print(
    var1 is True,
    var2 is True,
    var1 is var2,
    sep="\n",
)
# True
# True
# True
```

Логические представления

```
print(  
    bool(42),  
    bool(-3.14),  
    bool(0),  
    bool(None),  
    sep="\n",  
)  
# True  
# True  
# False  
# False
```

bool — подтип int

```
isinstance(True, int)           # True
```

```
True == 1                       # True
```

```
False == 0                      # True
```

```
True + True                     # 2
```

```
True + 1                        # 2
```

Синглтоны True и False

```
flag_literal = True
flag_from_bool = bool(1)
flag_from_comparison = (5 > 3)

print(
    flag_literal is flag_from_bool,
    flag_literal is flag_from_comparison,
    sep="\n",
)
# True
# True
```

Хороший тон сравнения с bool

Плохо

```
if is_valid == True:  
    ...  
  
if is_valid == False:  
    ...
```

Хорошо

```
if is_valid:  
    ...  
  
if not is_valid:  
    ...
```

Явные приведения в условиях

Плохо

```
items = get_items()

if bool(items):
    ...
```

Хорошо

```
items = get_items()

if items:
    ...
```

bool() уместен только как самостоятельный результат, а не внутри if/while



Арифметические операции

Унарный +

```
num = 42
```

```
print(  
    f"original num: {num}",  
    f"+num: {+num}",  
    num is +num,  
    sep="\n",  
)
```

```
# original num: 42
```

```
# +num: 42
```

```
# True
```

Унарный -

```
num = 42
```

```
print(  
    f"original num: {num}",  
    f"-num: {-num}",  
    num is -num,  
    sep="\n",  
)
```

```
# original num: 42
```

```
# -num: -42
```

```
# False
```

Бинарные арифметические операции

<code>3.14 + 5</code>	<code># сложение</code>
<code>5 - 3.14</code>	<code># вычитание</code>
<code>3.14 * 5</code>	<code># умножение</code>
<code>5 / 3.14</code>	<code># истинное деление (truediv)</code>
<code>5 // 3.14</code>	<code># целая часть от деления</code>
<code>5 % 3.14</code>	<code># остаток от деления</code>
<code>3.14 ** 5</code>	<code># возведение в степень</code>

Деление

```
>>> 1. / 2.  
0.5
```

```
>>> 1. / 2  
0.5
```

```
>>> 1 / 2.  
0.5
```

```
>>> 1 / 2  
0.5
```

Целочисленное деление

```
>>> 1 // 2
```

```
0
```

```
>>> 1. // 2
```

```
0.0
```

```
>>> 1 // 2.
```

```
0.0
```

```
>>> 1. // 2.
```

```
0.0
```

```
>>> -3 // 2
```

```
-2
```

Модульное деление

```
>>> 1 % 2
```

```
1
```

```
>>> 1. % 2
```

```
1.0
```

```
>>> 1 % 2.
```

```
1.0
```

```
>>> 1. % 2.
```

```
1.0
```

Особенности модульного деления

```
result = x - (y * (x // y))
```

```
>>> 5 % -3
```

```
-1
```

```
>>> -5 % 3
```

```
1
```

```
>>> -5 % -3
```

```
-2
```

Деление на ноль

```
>>> 1 / 0
```

```
ZeroDivisionError: division by zero
```

```
>>> 1 // 0
```

```
ZeroDivisionError: integer division or modulo by zero
```

```
>>> 1 % 0
```

```
ZeroDivisionError: integer modulo by zero
```

Возведение в степень

```
>>> 2 ** 3
```

```
8
```

```
>>> 2 ** -3
```

```
0.125
```

```
>>> 2 ** 0.5
```

```
1.4142135623730951
```

```
>>> (-1) ** 0.5
```

```
(6.123233995736766e-17+1j)
```

Погрешности float

```
0.1 + 0.2                # 0.30000000000000004
```

```
0.1 + 0.2 == 0.3         # False
```

```
abs((0.1 + 0.2) - 0.3) < 1e-9    # True -- верно
```

Приведение типов в операциях

<code>True + 1</code>	<code># 2</code>	<code>-- bool + int -> int</code>
<code>True + 1.0</code>	<code># 2.0</code>	<code>-- bool + float -> float</code>
<code>1 + 2.0</code>	<code># 3.0</code>	<code>-- int + float -> float</code>

Составные присваивания

без составного присваивания

```
num = 41
```

```
num = num + 1    # num == 42
```

```
num = 41
```

```
num += 1         # num == 42
```

Неизменяемость

```
num = 41  
print(id(num))    # 140710912780328
```

```
num += 1  
print(id(num))    # 140710912780360
```

Ограничения

переменная num не определена

num += 1

NameError: name 'num' is not defined.



Операции сравнения и логические операции

not

```
print(  
    not False,  
    not 42,  
    not -3.14,  
    not 0,  
    not float("nan"),  
    not float("inf"),  
    sep="\n",  
)  
# True  
# False  
# False  
# True  
# False  
# False
```

Специальная форма not

```
num = 42
```

```
# МОЖНО ТАК
```

```
if not (num is None):  
    print("num is not None!")
```

```
# НО ТАК ЛУЧШЕ
```

```
if num is not None:  
    print("num is not None!")
```

Логическое И

x	y	x and y
False	False	False
False	True	False
True	False	False
True	True	True

Operator and

```
print(  
    None and False,  
    0 and float("+inf"),  
    float("-inf") and 0,  
    42 and 3.14,  
    sep="\n",  
)  
# None  
# 0  
# 0  
# 3.14
```

Пример использования and

```
def do_expensive_computation() -> bool:  
    print("compute...")  
    return True
```

```
if False and do_expensive_computation():  
    ...  
# ничего не будет напечатано
```

```
if True and do_expensive_computation():  
    ...  
# compute...
```

Логическое ИЛИ

x	y	x or y
False	False	False
False	True	True
True	False	True
True	True	True

Operator or

```
print(  
    None or False,  
    0 or float("+inf"),  
    float("-inf") or 0,  
    42 or 3.14,  
    sep="\n",  
)  
# False  
# inf  
# -inf  
# 42
```

Операции сравнения

```
>>> 42 == 1337 # False
>>> 42 != 1337 # True
>>> 42 > 1337  # False
>>> 42 < 1337  # True
>>> 42 >= 1337 # False
>>> 42 <= 1337 # True
```

Цепочки операций

```
>>> 1 < 2 < 3 < 4
```

```
True
```

```
>>> 1 < 2 > 3
```

```
False
```

```
>>> 1 == 1.0 == True
```

```
True
```

```
>>> 1 is 1 == 1.0 == True
```

```
True
```

Эквиваленты

```
>>> 1 < 2 and 2 < 3 and 3 < 4  
True
```

```
>>> 1 < 2 and 2 > 3  
False
```

```
>>> 1 == 1.0 and 1.0 == True  
True
```

```
>>> 1 is 1 and 1 == 1.0 and 1.0 == True  
True
```

Вывод

```
# не будет ZeroDivisionError  
>>> 1 > 2 < (1 // 0)  
False
```

Мотивация использования

```
def compute() -> None:  
    print("compute")  
    return True
```

```
print(1 == compute() < 2)  
# compute  
print(1 == compute() and compute() < 2)  
# compute  
# compute
```



abs() и round()

abs() — модуль числа

```
abs(5)           # 5  
abs(-5)          # 5  
abs(-3.14)       # 3.14
```

```
distance = abs(7 - 2) # 5
```

`round()` — округление

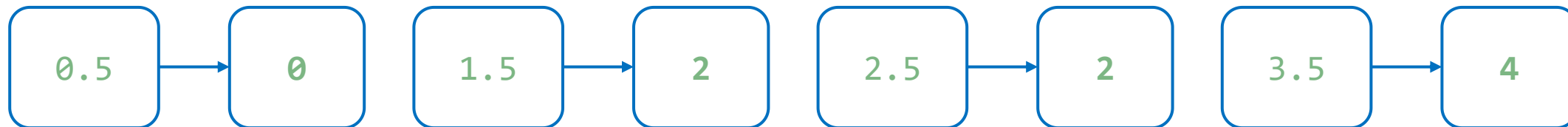
`round(3.2)` # 3

`round(-3.8)` # -4

`round(3.14159, 2)` # 3.14

`round(1234.5678, -2)` # 1200.0

round(): к ближайшему чётному



0 и 2 и 4 — чётные, поэтому округление идёт к ним



Спасибо за внимание!